

Python Cheat Sheet

ACSC430 — Dynamic Languages · Midterm topics

String slicing (Q1)

Syntax

string[start:end] — omit end slices to end of string

```
s = '1357 Country Ln.'
s[4:] -> ' Country Ln.'
s[:4] -> '1357'
s[4:8] -> ' Cou'
```

String methods (Q8, Q10)

Strip chars from the END

str.rstrip('chars')

Strip chars from the START

str.lstrip('chars')

Check if substring present

str.find(substring) — returns index or -1

Loop control (Q4, Q16, Q17)

Loop that repeats based on boolean expression

Condition-controlled loop (while)

break — stop when condition met

```
for x in fruits:
    if x == 'banana':
        break
print(x) # apple
```

continue — skip this iteration, carry on

```
for x in fruits:
    if x == 'banana':
        continue
print(x) # apple, cherry
```

Iterate all items

```
for x in fruits:
    print(x) # apple, banana, cherry
```

Conditionals & compound expressions (Q5, Q6)

Check if choice is anything other than 10

if choice != 10: (colon required)

Combining multiple boolean expressions

Use and / or / not -> creates compound expressions

Lists (Q14)

Remove element at a specific index

del list[index] statement

Remove by value

list.remove(value) method

Key difference

del -> by position; remove() -> by value

OOP — classes (Q11, Q19, Q20, Q21)

Method auto-executed when instance is created

__init__(self, ...)

String representation of an object

__str__(self) — called by print(obj)

Private attributes (encapsulation)

Prefix with __ e.g. self.__name

Inheritance syntax

```
class Student(Person):
    pass
x = Student('Mike', 'Moor')
x.printname() # Mike Moor
```

Call parent __init__ from child

```
class Child(Parent):
    def __init__(self):
        super().__init__(args)
```

Person / Client class pattern (Q21)

```
class Person:
    def __init__(self, name, email, mobile):
        self.__name = name
        self.__email = email
        self.__mobile = mobile
    def get_name(self): return self.__name
    def set_name(self, n): self.__name = n
    def __str__(self):
        return (f'Name: {self.__name}\n'
                f'Address: {self.__email}\n'
                f'Phone: {self.__mobile}')

class Client(Person):
    def __init__(self, name, email, mobile, cnum, sms):
        super().__init__(name, email, mobile)
        self.__client_number = cnum
        self.__sms_promo = sms
    def __str__(self):
        return (f'{super().__str__()\n'
                f'Customer Number: {self.__client_number}\n'
                f'On SMS List: {self.__sms_promo}')
```

Plant / Tree output trace (Q20)

```
p = Plant('lemon')
t = Tree() # Plant.__init__(self, 'tree')
p.message() # I'm a plant.
t.message() # I'm a tree. I am tree
print(p) # I am a lemon (via __str__)
```

Key concept: method overriding

Child message() overrides parent's — Python calls the most specific version

Dictionaries (Q7)

Accessing a key that does not exist

Raises KeyError

```
ages = {'Aaron': 6, 'Kelly': 3}
ages['Brianna'] # KeyError!
ages.get('Brianna', 0) # safe: returns 0
```

File I/O & variables (Q3, Q9, Q13)

Process of retrieving data from a file

Reading data

First input before a validation loop

Priming read

Variable accessible to all functions in a file

Global variable — use global varname inside function to modify

Turtle graphics (Q2, Q15)

Import statement

import turtle — lowercase, no suffix

```
turtle.penup()
turtle.goto(100, 0)
turtle.fillcolor('blue')
turtle.pendown()
turtle.begin_fill()
for i in range(4):
    turtle.forward(50)
    turtle.left(90)
turtle.end_fill()
```

Misc (Q12, Q18)

Diagram graphically depicting program steps

Flowchart (not pseudocode, not algorithm)

Correct string formatting

```
print(f'Amount: ${amount:.2f}')
print('Amount: ${:.2f}'.format(amount))
```